

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	Krithika M	Reg. No.:	192521385
Section / Batch:	B.Tech IT	Date:	10:08:2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I Krithika M 192521385Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **Krithika M** _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Rubrics:

Criteria	Weightage	Awarded Marks	Total Marks (100)
Understanding of Concepts	25%		
Experimental Design	30%		
Use of Tools	20%		
Analysis of Results	15%		
Documentation	10%		
Total per Question	100 Marks		

— 100

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name: Krithika M

Course name: DevSecOps

Real-world problem: Vulnerable Online Banking Application

An online banking application contains source-code modules such as login, fund transfer, user profile, and transaction search. Security problems can occur because of SQL injection, hard-coded passwords, weak authentication, insecure input handling, etc.

Instead of using confidential real banking code, you create synthetic source-code samples that imitate these situations. Your SAST simulator scans the synthetic code and identifies vulnerabilities.

Proposed project flow

Synthetic Source Code Dataset



Preprocessing / Code Parsing



SAST Rule Engine



Vulnerability Detection



Severity Classification



Risk Score Calculation



Secure Code Review Report

Synthetic dataset

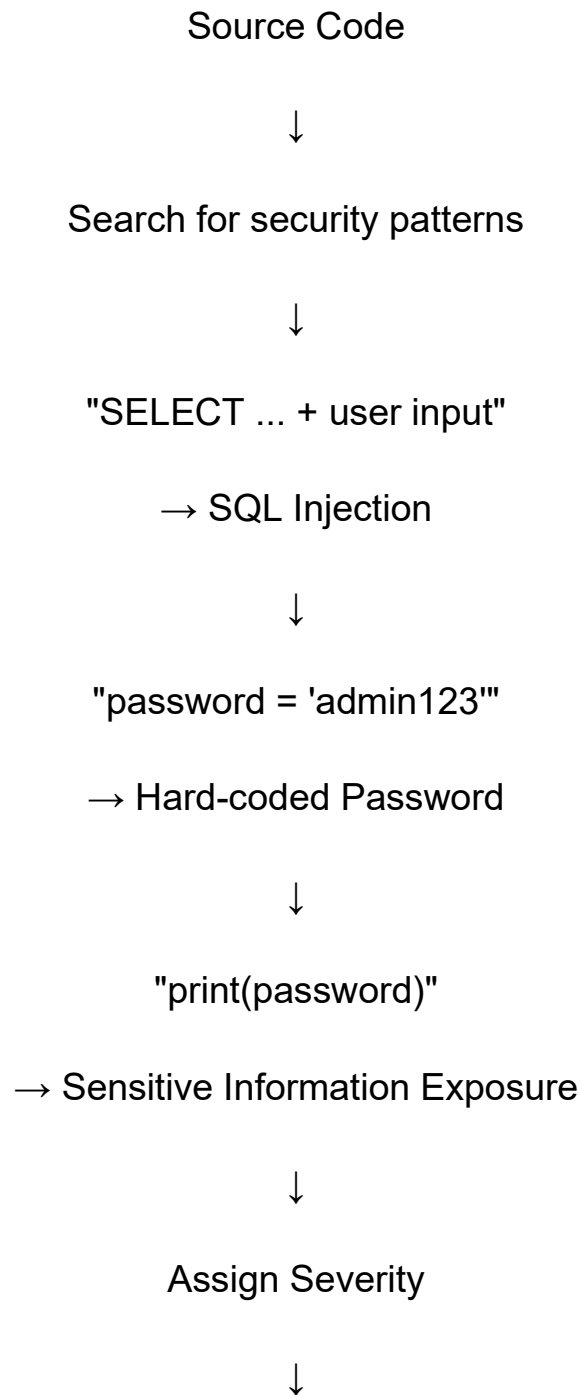
ID	Module	Code Pattern	Vulnerability	Severity	SAST Result
1	Login	SQL query uses user input directly	SQL Injection	Critical	Detected
2	Login	Password stored directly in code	Hard-coded Password	High	Detected
3	Fund Transfer	User input not validated	Input Validation	High	Detected
4	Profile	Uses parameterized query	No vulnerability	Safe	Not Detected
5	Transaction	Sensitive data printed in logs	Information Exposure	Medium	Detected
6	Login	Weak password comparison	Weak Authentication	High	Detected
7	Transfer	Amount accepted without validation	Improper Validation	Medium	Detected
8	Profile	Parameterized SQL query	No vulnerability	Safe	Not Detected
9	Transaction	Database credentials exposed	Hard-coded Credential	Critical	Detected
10	Login	Input sanitized before query	No vulnerability	Safe	Not Detected

You can create a CSV dataset like this:

How you simulate SAST

Your implementation doesn't need a real commercial SAST tool. You can build a rule-based SAST simulator in Python.

For example:



Generate Security Report

Example output

Suppose the synthetic code contains:

```
username = input ("Enter username: ")
```

```
query = "SELECT * FROM users WHERE name = " + username + ""
```

Your simulated SAST engine detects:

Vulnerability: SQL Injection

Severity: Critical

Module: Login

Detection: Direct user input used in SQL query

Recommendation: Use parameterized queries

Where AI comes into your project

To make it fit “AI-Powered”, you can add an AI/ML component after the SAST rule detection:

Code Features → ML Model → Vulnerability Prediction → Severity

For example, train a simple classifier using your synthetic dataset to predict:

- Safe
- Low
- Medium
- High

- Critical

So project has two parts:

Module 1 – SAST Vulnerability Detection Engine

Detects known insecure coding patterns using static-analysis rules.

Module 2 – AI-Based Risk Assessment & Secure Code Review

Uses the detected features to predict vulnerability severity and generate recommendations.

This is a very practical project because you can implement the entire demonstration using Python + synthetic data, without needing access to confidential real-world source code.

**SecureScan AI**
AI-Powered SAST Platform

[Dashboard](#)
[Scan Code](#)
[Reports](#)
[Vulnerabilities](#)
[Remediation](#)
[Settings](#)
[Help](#)

Scan Summary
Project
Synthetic Web App

Language
Python

Scan Type
Static Application
Security Testing (SAST)

Date & Time
12 May 2025, 10:30 AM

Status
Completed

SAST Scan Results

Scan ID: SCAN_20250512_1030 [Download Report](#)

Overall Risk Score
73 / 100
High Risk


Vulnerabilities

Critical	High	Medium	Low
3	7	5	2

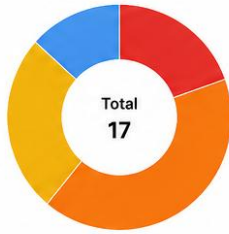
Files Scanned
12 / 12
100% Scanned


Lines of Code
1,247
Python


Top Vulnerabilities

ID	Vulnerability Type	Severity	File	Line
VULN-001	SQL Injection	Critical	app.py	23
VULN-002	Hardcoded Secret	Critical	config.py	8
VULN-003	Command Injection	High	utils.py	47
VULN-004	Insecure Deserialization	High	handlers.py	112
VULN-005	XSS (Reflected)	Medium	search.py	31

[View all vulnerabilities](#)

Vulnerability Distribution

Severity	Count	Percentage
Critical	3	18%
High	7	41%
Medium	5	29%
Low	2	12%

Code Snippet (app.py : Line 23)

```
20
21 def get_user(user_id):
22     query = "SELECT * FROM users WHERE id = " + user_id + " "
23     result = cursor.execute(query)      ← Vulnerable: SQL Injection
24     return result.fetchall()
25
26 # More code...
```

Issue: User input is directly concatenated into SQL query.

AI Recommendation
 **Fix: Use Parameterized Query**
Why?
Parameterized queries prevent SQL injection by separating SQL code from user input.
Example:

```
query = "SELECT * FROM users WHERE id = ?"
result = cursor.execute(query, (user_id,))
```

[Apply Fix \(Auto\)](#)

This is a simulated SAST scan output using synthetic data for demonstration purposes.